Experiment 3: Bubble Sort Visualization Tool

Aim

To implement the **Bubble Sort** algorithm in Python and display the array after every pass to visualize the sorting process.

Problem Statement

Develop a Bubble Sort program that prints the state of the array after each pass. This helps students understand how the largest element moves to its correct position in every pass.

Algorithm

Step 1: Start.

Step 2: Read the number of elements.

Step 3: Read the array elements.

Step 4: Apply Bubble Sort.

Step 5: After every pass, display the current array.

Step 6: Repeat until the array is sorted.

Step 7: Stop.

Source Code (Python)

main.py

```python
# Bubble Sort Visualization

n = int(input("Enter number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

    print("After Pass", i + 1, ":", arr)

print("Sorted Array:", arr)
```

Output

```
Output
Enter number of elements: 5
Enter the elements:
5
1
4
2
8
After Pass 1 : [1, 4, 2, 5, 8]
After Pass 2 : [1, 2, 4, 5, 8]
After Pass 3 : [1, 2, 4, 5, 8]
After Pass 4 : [1, 2, 4, 5, 8]
Sorted Array: [1, 2, 4, 5, 8]

=== Code Execution Successful ===
```

Complexity Analysis

Aspect	Complexity
Time Complexity	**$O(n^2)$**
Space Complexity	**O(1)**

Result

The Bubble Sort algorithm was successfully implemented, and the array was displayed after every pass, making the sorting process easy to visualize.